

RL Snake Using Hamiltonian Cycles

By: Nivedh Biju (CS22B1021)

Nu Aim Mohammed Chemnad Lasiyath (CS22B1024)

Project Overview:

- This project redefines the classic reinforcement learning (RL) approach to the Snake game by aiming not just for apple collection, but complete grid traversal through Hamiltonian cycles.
- Instead of reactive, reward-driven behaviors, the snake learns to systematically cover all grid cells—ensuring survival, efficiency, and long-term strategy.

Problem Statement:

Train a snake agent that can traverse every cell in an $N \times N$ grid exactly once and return to the starting point—forming a Hamiltonian cycle. This differs from traditional methods that reward short-term goals like eating apples.

Motivation:

- Apple-centric strategies often lead to
 - Poor generalization to larger
 - Self-trapping
 - Failure to fully explore the
- In contrast, Hamiltonian cycles:
 - Ensure full coverage
 - Guarantee no collisions
 - Scale to real-world path planning problems (e.g., vacuum robots, drones)

Environment Design:

- Grid-based world: Grids range from 2×2 to 6×6 , with the potential to expand further
- State representation: Binary matrix or multi-channel tensors, used as input to a CNN-based DQN
- Action space: 4 discrete directions (Up, Down, Left, Right). Illegal moves are either filtered or penalized.

Learning Approaches:

1. Base RL Snake

- Trained directly on a fixed grid size (e.g., 6×6) using a CNN-based DQN architecture
- Basic reward structure:
 - +1 for unvisited
 - -1 for visited
 - +10 for completing a cycle
 - Episode ends on collision
- Challenges:
 - Often fails to complete the cycle
 - Learns to fill the grid, but lacks planning to return to start

Hyperparameter	Value
Grid Size	6 × 6
Episodes	5000
Learning Rate	0.01
Gamma (γ)	0.9
Epsilon Range	100 → 0.1
Epsilon Decay	0.995
Replay Buffer Size	2000
Batch Size	64
Optimizer	Adam
Save Frequency	Every 100 episodes

○ Results:

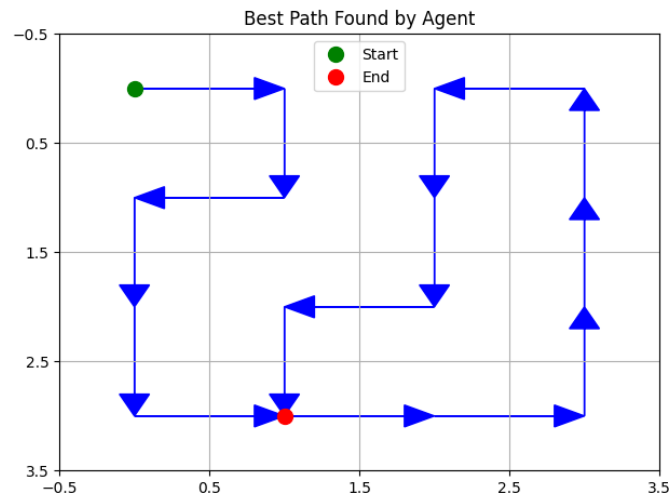


Figure 1: 4x4 Grid



Figure 2: 6x6 Grid

2. Transfer Learning & Curriculum Strategy

○ Curriculum Learning Pipeline:

- Train on 2x2 grid → Save weights
- Transfer to 4x4 grid → Fine-tune
- Scale to 6x6 grid using learned

○ Dense reward structure:

- +100: new cell
- -1000: crash or revisit
- $+1000 \times (\text{visited_ratio})$: partial return to start

- +10000: full Hamiltonian cycle

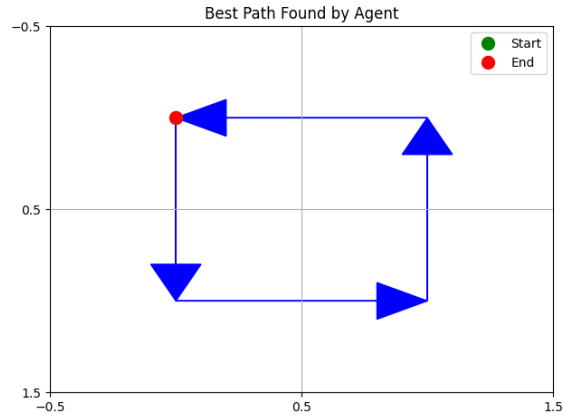
- CNN-DQN Architecture:

- Two convolutional layers + two dense layers
- Epsilon-greedy strategy with experience replay
- Transfer of weights between grid sizes

Parameter	Value
GRID_SIZE	2 (initial)
EPSILON	100
EPSILON_DECAY	0.995
EPSILON_MIN	0.1
GAMMA	0.9
LEARNING_RATE	0.01
BATCH_SIZE	64
MEMORY_SIZE	2000
EPISODES	1500

- Results:

- Agent successfully learns Hamiltonian cycles on the 6×6 grid
- Clear reuse of learned paths from smaller grids



- **Figure 3: 2x2 Grid**



Figure 4: 4x4 Grid



Figure 5: 6x6 Grid

Key Observations:

- Transfer learning is critical
- Spatial patterns generalize
- Dense rewards accelerate learning

Challenges:

- Balancing two competing objectives
 1. Fill all grid
 2. Return to the start without revisiting
- Without curriculum learning, models struggle with long-term planning
- High dimensionality of action/state space on larger grids increases sample inefficiency

Why CNNs?

- Spatial invariance: Local feature detection improves movement
- Efficiency: Fewer parameters than dense networks
- Transferability: Works across variable input sizes

Conclusions:

- Hamiltonian cycles provide a principled framework for solving the Snake game
- Curriculum learning and reward shaping enable the agent to achieve long-term goals
- The strategy extends to real-world applications in robotics, navigation, and logistics

Work Distribution:

- Nivedh -> CNN model for DQN, Transfer Learning/Curriculum Strategy
- Nuaim -> Building the environment, Testing with ANN model for DQN

References:

- [An Introduction to Q-Learning \(https://www.freecodecamp.org/news/an-introduction-to-q-learning-reinforcement-learning-14ac0b4493cc/\)](https://www.freecodecamp.org/news/an-introduction-to-q-learning-reinforcement-learning-14ac0b4493cc/)
- [Proximal Policy Optimization in RL \(https://www.geeksforgeeks.org/a-brief-introduction-to-proximal-policy-optimization/\)](https://www.geeksforgeeks.org/a-brief-introduction-to-proximal-policy-optimization/)
- [Deep Q-Learning \(https://www.geeksforgeeks.org/deep-q-learning/\)](https://www.geeksforgeeks.org/deep-q-learning/)
- [M. Rahman, M. Kaykobad, On Hamiltonian cycles and Hamiltonian paths, Information Processing Letters 94 \(2005\)](#)
- [Ruikang Zhang, Ruikai Cai, Train a snake with reinforcement learning algorithms, Department of Information Engineering The Chinese University of Hong Kong Shatin](#)